



Python Programming Fundamentals

Sorting Lists of Objects

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. **sort() vs sorted()**
2. **Sorting Strings**
3. **The key Parameter**
4. **Sorting Objects with key=**
5. **lambda Functions**
6. **Sorting with Multiple Keys**
7. **Adding Sort to Shop**



Outline

- 1. `sort()` vs `sorted()`**
2. Sorting Strings
3. The key Parameter
4. Sorting Objects with `key=`
5. lambda Functions
6. Sorting with Multiple Keys
7. Adding Sort to Shop



1. sort() vs sorted()

<code>list.sort()</code>	<code>sorted(iterable)</code>
Modifies the list in place	Returns a new sorted list
Returns None	Original list unchanged
Only works on lists	Works on any iterable

```
numbers = [3, 1, 4, 1, 5, 9, 2, 6]
```

```
numbers.sort()           # modifies numbers
print(numbers)           # [1, 1, 2, 3, 4, 5, 6, 9]
```

```
result = sorted(numbers, reverse=True)
print(result)             # [9, 6, 5, 4, 3, 2, 1, 1]
print(numbers)            # unchanged
```



Outline

1. `sort()` vs `sorted()`
2. **Sorting Strings**
3. The key Parameter
4. Sorting Objects with `key=`
5. `lambda` Functions
6. Sorting with Multiple Keys
7. Adding Sort to Shop



2. Sorting Strings

```
names = ["Milk", "Apple", "Bread", "Butter"]
```

```
names.sort()           # alphabetical, in place  
print(names)           # ['Apple', 'Bread', 'Butter', 'Milk']
```

```
sorted_names = sorted(names, reverse=True)  
print(sorted_names)    # ['Milk', 'Butter', 'Bread', 'Apple']
```

```
# Case-insensitive sort  
words = ["banana", "Apple", "cherry"]  
words.sort(key=str.lower)  
print(words)           # ['Apple', 'banana', 'cherry']
```



Outline

1. `sort()` vs `sorted()`
2. Sorting Strings
- 3. The key Parameter**
4. Sorting Objects with `key=`
5. `lambda` Functions
6. Sorting with Multiple Keys
7. Adding Sort to Shop



3.1 Sorting by a function

```
# key= takes a function that extracts the sort value
numbers = [3, -1, 4, -2, 5]
numbers.sort(key=abs)           # sort by absolute value
print(numbers)                  # [-1, -2, 3, 4, 5]

words = ["fig", "banana", "apple", "date"]
words.sort(key=len)             # sort by length
print(words)                    # ['fig', 'fig', 'date', 'apple', 'banana']
```



Outline

1. `sort()` vs `sorted()`
2. Sorting Strings
3. The key Parameter
- 4. Sorting Objects with `key=`**
5. lambda Functions
6. Sorting with Multiple Keys
7. Adding Sort to Shop



4.1 Using lambda

```
products = shop.get_all_products()

# Sort by price (cheapest first)
by_price = sorted(products, key=lambda p: p.get_price())

# Sort by name (A-Z)
by_name = sorted(products, key=lambda p: p.get_name().lower())

# Sort by price descending (most expensive first)
by_price_desc = sorted(products,
                        key=lambda p: p.get_price(),
                        reverse=True)

for p in by_price:
    print(f"    €{p.get_price():.2f}    {p.get_name()}")
```



Outline

1. `sort()` vs `sorted()`
2. Sorting Strings
3. The key Parameter
4. Sorting Objects with `key=`
5. **lambda Functions**
6. Sorting with Multiple Keys
7. Adding Sort to Shop



5.1 Anonymous, one-expression functions

Regular function

```
def get_price(product):  
    return product.get_price()
```

Equivalent lambda

```
get_price = lambda product: product.get_price()
```

Used inline – no need to define separately

```
sorted(products, key=lambda p: p.get_price())
```

lambda arguments: expression

Lambda is ideal for **short, single-use** functions — especially as key= arguments.



Outline

1. `sort()` vs `sorted()`
2. Sorting Strings
3. The key Parameter
4. Sorting Objects with `key=`
5. `lambda` Functions
- 6. Sorting with Multiple Keys**
7. Adding Sort to Shop



6. Sorting with Multiple Keys

```
students = [  
    ("Alice", "B", 20),  
    ("Bob", "A", 22),  
    ("Carol", "B", 19),  
    ("Dave", "A", 20),  
]  
  
# Sort by grade first, then by name  
students.sort(key=lambda s: (s[1], s[0]))  
# [('Bob', 'A', 22), ('Dave', 'A', 20),  
#  ('Alice', 'B', 20), ('Carol', 'B', 19)]
```

For objects:

```
# Sort by category, then by price  
products.sort(key=lambda p: (
```



6. Sorting with Multiple Keys

```
p.get_category(),  
p.get_price()  
)
```



Outline

1. `sort()` vs `sorted()`
2. Sorting Strings
3. The key Parameter
4. Sorting Objects with `key=`
5. lambda Functions
6. Sorting with Multiple Keys
- 7. Adding Sort to Shop**



7. Adding Sort to Shop

```
# In shop.py
def get_sorted_by_price(self, reverse: bool = False) -> list:
    return sorted(self.__products,
                  key=lambda p: p.get_price(),
                  reverse=reverse)

def get_sorted_by_name(self) -> list:
    return sorted(self.__products,
                  key=lambda p: p.get_name().lower())

# In driver.py
print("\nSorted by price:")
for p in shop.get_sorted_by_price():
    print(f"    {p}")

print("\nSorted by name:")
```



7. Adding Sort to Shop

```
for p in shop.get_sorted_by_name():  
    print(f"    {p}")
```

Thanks for Watching - Any questions?

